



Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology (BUET)

Offline-1: AES + Diffie-Hellman Cryptosystem

Design, Theory, and Performance Analysis of an End-to-End
Symmetric-Key Hybrid Cryptosystem

Course Title: CSE 406 — Cyber Security Sessional
Academic Term: Level 4, Term 1 (January 2026)
Assignment Reference: Offline-1 (Cryptography)

Submitted By:

Nawriz Ahmed Turjo

Student ID: 2105032

Section: A2

Submitted on: July 14, 2026

Contents

1	Introduction	1
1.1	End-to-End System Architecture	1
2	Theoretical Background and Mathematical Formulations	3
2.1	Diffie-Hellman Key Exchange Protocol	3
2.1.1	Protocol Execution Flow	3
2.1.2	Miller-Rabin Primality Testing	3
2.1.3	Safe Prime Criteria and Generator Selection	4
2.2	Advanced Encryption Standard (AES)	4
2.2.1	Finite Field Arithmetic in $GF(2^8)$	4
2.2.2	Round Transformations	5
2.2.3	Key Expansion Schedule	6
2.3	Modes of Operation	6
2.3.1	Electronic Codebook (ECB) Mode	6
2.3.2	Cipher Block Chaining (CBC) Mode	6
2.3.3	PKCS#7 Padding	7
3	Code Architecture and Design Choices	8
3.1	Module Inventory	8
3.2	Design Choices and Optimization Strategies	8
3.2.1	Optimized Galois Field Look-Up Table (LUT)	8
3.2.2	Key Normalization and Expansion Supporting 128, 192, and 256 Bits	9
3.2.3	Secure Session Key Derivation	10
3.2.4	Socket Network Architecture	10
3.2.5	Bitmap Structure Preservation	11
4	Experimental Results and Performance Analysis	12
4.1	AES Performance Benchmarks	12
4.2	Diffie-Hellman Parameter Generation Benchmarks	12
4.3	Analysis of Galois Field Look-Up Table (GF LUT) Optimization	13
4.3.1	Theoretical Complexity and Overhead Analysis	14
5	Security Evaluation and Design Rationale	16

5.1	Visual Cryptanalysis: ECB vs. CBC on BMP Images	16
5.1.1	Why ECB Leaks Visual Outlines	16
5.1.2	How CBC Prevents Pattern Leakage	16
5.2	Vulnerability Analysis and Mitigations	17
5.2.1	Proposed Mitigations	18
6	Conclusion	18

1 Introduction

Modern information security relies heavily on **hybrid cryptosystems** to balance safety and performance. While public-key (asymmetric) cryptosystems solve the primary bottleneck of secure key transport over insecure networks, they are computationally intensive. Conversely, symmetric-key ciphers are orders of magnitude faster but require a pre-shared secret key.

This project details the independent implementation of a secure symmetric-key hybrid cryptosystem from first principles:

1. **Diffie-Hellman Key Exchange:** A secure, modular arithmetic key-exchange protocol executed over a TCP socket channel, enabling two untrusted endpoints (**Alice** and **Bob**) to establish a shared cryptographic secret. **Miller-Rabin primality testing** and **safe prime generators** ensure resistance to discrete logarithm attacks.
2. **Advanced Encryption Standard (AES):** A symmetric block cipher implementation executing the standard round transformations (*SubBytes*, *ShiftRows*, *MixColumns*, and *AddRoundKey*) in column-major order to achieve strong confusion and diffusion properties.
3. **Modes of Operation & Transport:** Encryption and decryption of multi-block messages in both **Electronic Codebook (ECB)** and **Cipher Block Chaining (CBC)** configurations (with proper **PKCS#7 padding**), streaming textual payloads and arbitrary binary files over network interfaces.

The system components are implemented natively in Python without relying on high-level cryptography suites (using only low-level math primitives and helper constants for finite field mappings).

1.1 End-to-End System Architecture

To illustrate the interaction between the sender (**Alice**) and the receiver (**Bob**), the following sequence mapping outlines the hybrid system lifecycle:

1. **Parameter Generation (Step 1):** Alice generates public parameters locally using Miller-Rabin testing to produce a safe prime P (where $P = 2q + 1$ and q is prime) and finding a generator g for the multiplicative subgroup of order q .
2. **Key Exchange (Step 2):** Alice generates a private exponent K_a , computes public key $A = g^{K_a} \bmod P$, and transmits the tuple (P, g, A) over the TCP socket. Bob accepts, generates his private exponent K_b , computes public key $B = g^{K_b} \bmod P$, and transmits B back to Alice.
3. **Key Derivation (Step 3):** Both parties calculate the shared secret s independently:

$$s = B^{K_a} \bmod P \quad (\text{Alice}) \quad \text{and} \quad s = A^{K_b} \bmod P \quad (\text{Bob}) \quad (1)$$

A 128-bit session key K is derived from s by taking its big-endian byte representation.

4. **Secure Session Transmission (Step 4):** Alice encrypts the payload (text or binary file bytes) in CBC mode using K and a randomly generated IV. She formats the transmission using a 4-part protocol header frame:

[Message Type : 4B] || [Filename Length : 4B] || [Filename : Var] || [Ciphertext Length : 10B]

The ciphertext stream is sent once Bob signals he is "READY". Bob receives the header, reads the expected ciphertext length, decrypts the block, and strips the PKCS#7 padding.

The sequence flow is visualized in Figure 1.

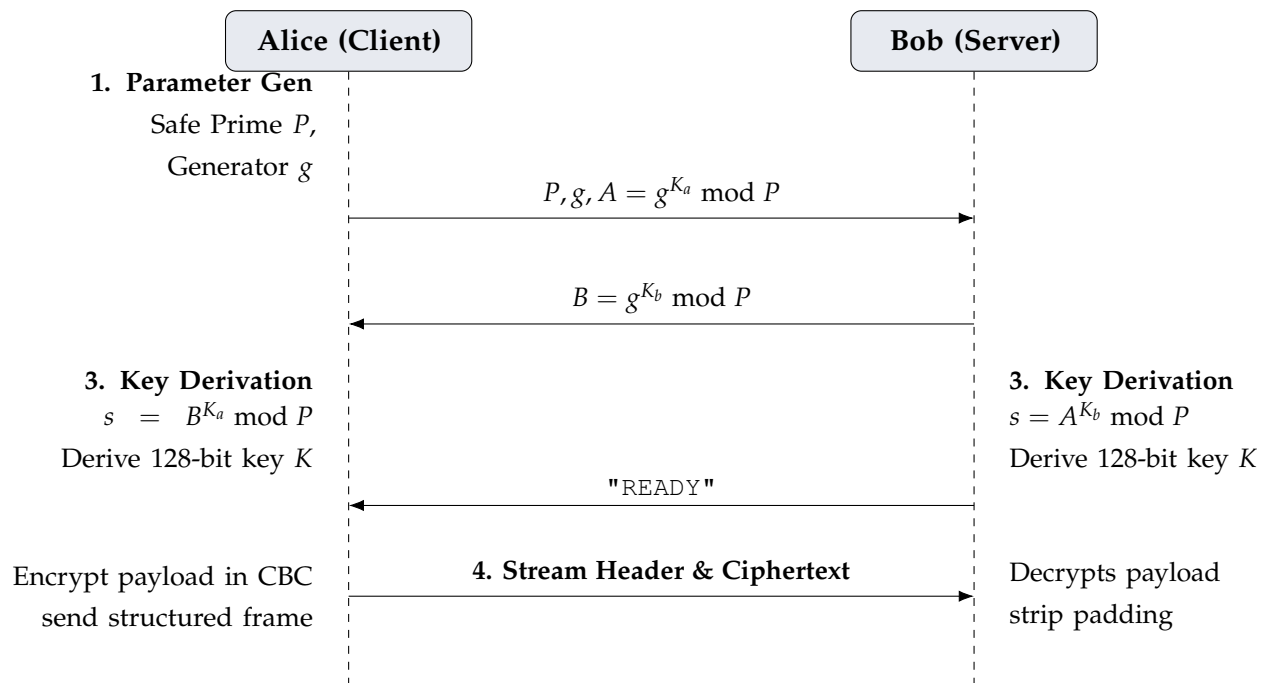


Figure 1: End-to-End Hybrid Cryptosystem Sequence Flow Diagram.

2 Theoretical Background and Mathematical Formulations

2.1 Diffie-Hellman Key Exchange Protocol

The Diffie-Hellman (DH) protocol relies on the computational difficulty of the **Discrete Logarithm Problem (DLP)** in multiplicative groups of finite fields. Given the public values $g^x \bmod P$, g , and P , determining the exponent x is computationally infeasible for a sufficiently large prime P .

2.1.1 Protocol Execution Flow

Alice and Bob negotiate parameters publicly and compute key values:

1. **Parameter Setup:** A large prime P and a generator g of the multiplicative group $\mathbb{Z}_P^*$ (where $1 < g < P$) are generated.

2. **Private/Public Key Generation:**

- Alice generates a random secret exponent $K_a \geq k$ -bits and computes:

$$A = g^{K_a} \bmod P \quad (2)$$

- Bob generates a random secret exponent $K_b \geq k$ -bits and computes:

$$B = g^{K_b} \bmod P \quad (3)$$

3. **Key Negotiation:** Alice transmits A to Bob, and Bob transmits B to Alice.

4. **Shared Secret Derivation:**

- Alice computes:

$$s = B^{K_a} \bmod P \quad (4)$$

- Bob computes:

$$s = A^{K_b} \bmod P \quad (5)$$

Both compute the same value because:

$$s = (g^{K_b})^{K_a} \equiv (g^{K_a})^{K_b} \equiv g^{K_a K_b} \pmod{P} \quad (6)$$

2.1.2 Miller-Rabin Primality Testing

To guarantee that P is prime, a probabilistic primality test is used. The Miller-Rabin test begins by writing $P - 1$ as:

$$P - 1 = 2^s \cdot d \quad \text{where } d \text{ is odd} \quad (7)$$

For a chosen base $a \in [2, P - 2]$, we check if:

$$a^d \equiv 1 \pmod{P} \quad \text{or} \quad a^{2^r \cdot d} \equiv P - 1 \pmod{P} \quad \text{where } r \in [0, s - 1] \quad (8)$$

If neither condition holds, the candidate P is composite. Repeating this check with N independent random bases reduces the false-positive probability to less than 4^{-N} . For security, $N = 40$ iterations were selected.

2.1.3 Safe Prime Criteria and Generator Selection

A prime P is a **safe prime** if it satisfies:

$$P = 2q + 1 \quad (9)$$

where q is also prime (a Sophie Germain prime). Using a safe prime prevents attacks like the **Pohlig-Hellman algorithm**, which exploits prime factors of $P - 1$. In a safe prime, $P - 1 = 2q$, meaning the only prime factors of $P - 1$ are 2 and q .

To find a generator $g \in [2, P - 2]$ for the subgroup of order q or $2q$ in $\mathbb{Z}_P^*$, we ensure that for all prime factors r of $P - 1$:

$$g^{(P-1)/r} \not\equiv 1 \pmod{P} \quad (10)$$

Since the only prime factors of $P - 1$ are $\{2, q\}$, generator validation simplifies to testing:

$$g^2 \not\equiv 1 \pmod{P} \quad \text{and} \quad g^q \not\equiv 1 \pmod{P} \quad (11)$$

If g meets both criteria, it is confirmed as a valid generator.

2.2 Advanced Encryption Standard (AES)

AES is a symmetric-key block cipher that operates on a 4×4 **state matrix** of bytes in column-major order. Let a 16-byte block be $B = [b_0, b_1, \dots, b_{15}]$. The state matrix S is organized as:

$$S = \begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix} \quad (12)$$

2.2.1 Finite Field Arithmetic in $\text{GF}(2^8)$

AES addition and multiplication are performed in the Galois Field $\text{GF}(2^8)$ defined by the irreducible polynomial:

$$P(x) = x^8 + x^4 + x^3 + x + 1 \quad (13)$$

- **Addition:** Addition of two byte values $A(x) + B(x)$ corresponds to the bitwise exclusive-OR (XOR) operation:

$$A \oplus B \quad (14)$$

- **Multiplication:** Multiplication of $A(x) \cdot B(x)$ is computed modulo $P(x)$. This is per-

formed efficiently using binary shifts and reductions, known as the **xtime** operation:

$$x \cdot A(x) \pmod{P(x)} = \begin{cases} A(x) \ll 1 & \text{if the MSB of } A \text{ is } 0 \\ (A(x) \ll 1) \oplus 0x1B & \text{if the MSB of } A \text{ is } 1 \end{cases} \quad (15)$$

2.2.2 Round Transformations

Each intermediate round $i \in [1, N_r - 1]$ consists of four steps:

1. **SubBytes**: A non-linear byte substitution. Each byte $S_{r,c}$ in the state is mapped to its multiplicative inverse in $\text{GF}(2^8)$, followed by an affine transformation over $\text{GF}(2)$:

$$S_{r,c} \leftarrow \text{Sbox}[S_{r,c}] \quad (16)$$

2. **ShiftRows**: A cyclic shift applied to the rows of the state. Row r (indexed $0 \leq r \leq 3$) is cyclically shifted left by r bytes:

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \xrightarrow{\text{ShiftRows}} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix} \quad (17)$$

3. **MixColumns**: A linear transformation applied to each column of the state matrix. Each column is multiplied by a fixed polynomial matrix over $\text{GF}(2^8)$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad (18)$$

All multiplications and additions are evaluated within $\text{GF}(2^8)$. For decryption, the inverse matrix is used:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0x0E & 0x0B & 0x0D & 0x09 \\ 0x09 & 0x0E & 0x0B & 0x0D \\ 0x0D & 0x09 & 0x0E & 0x0B \\ 0x0B & 0x0D & 0x09 & 0x0E \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad (19)$$

4. **AddRoundKey**: The state matrix is XORed with the corresponding round key $W_{4i \dots 4i+3}$:

$$S_{r,c} \leftarrow S_{r,c} \oplus K_{r,c}^{(i)} \quad (20)$$

In the final round N_r , the **MixColumns transformation is omitted**.

2.2.3 Key Expansion Schedule

The key schedule expands the original cipher key K into $N_r + 1$ round keys. The expansion uses 32-bit words, producing $4(N_r + 1)$ total words. For a key length N_k (in 32-bit words), the words $w[i]$ are derived as follows:

- For $i < N_k$, the words are copied directly from the key.
- For $i \geq N_k$, the formula depends on $i \bmod N_k$:

$$w[i] = \begin{cases} w[i - N_k] \oplus g(w[i - 1]) & \text{if } i \equiv 0 \pmod{N_k} \\ w[i - N_k] \oplus \text{SubWord}(w[i - 1]) & \text{if } N_k = 8 \text{ and } i \equiv 4 \pmod{N_k} \\ w[i - N_k] \oplus w[i - 1] & \text{otherwise} \end{cases} \quad (21)$$

The g -function processes a 4-byte word:

1. **RotWord**: Rotates the word cyclically by 1 byte to the left:

$$[a_0, a_1, a_2, a_3] \xrightarrow{\text{RotWord}} [a_1, a_2, a_3, a_0] \quad (22)$$

2. **SubWord**: Substitutes each byte using the AES S-box lookup table.
3. **Rcon XOR**: XORs the first byte with the round constant $Rcon[j]$:

$$Rcon[j] = [x^{j-1} \bmod P(x), 0, 0, 0] \quad (23)$$

2.3 Modes of Operation

Block ciphers encrypt fixed-length blocks. To process arbitrary-length data, the following modes are used:

2.3.1 Electronic Codebook (ECB) Mode

ECB mode encrypts each block of plaintext independently:

$$C_i = E_K(P_i) \quad \text{and} \quad P_i = D_K(C_i) \quad (24)$$

Because identical plaintext blocks map to identical ciphertext blocks under the same key, ECB does not hide data patterns. This makes it unsuitable for most cryptographic applications.

2.3.2 Cipher Block Chaining (CBC) Mode

CBC mode introduces an Initialization Vector (IV) and chains blocks to ensure that identical plaintext blocks encrypt to different ciphertext blocks:

$$C_i = E_K(P_i \oplus C_{i-1}) \quad \text{with } C_0 = IV \quad (25)$$

During decryption, each block is decrypted and then XORed with the previous ciphertext block:

$$P_i = D_K(C_i) \oplus C_{i-1} \quad \text{with } C_0 = IV \quad (26)$$

2.3.3 PKCS#7 Padding

Because AES operates on 128-bit (16-byte) blocks, plaintext must be padded to a multiple of 16 bytes. PKCS#7 padding appends L bytes, each containing the value of L , where:

$$L = 16 - (\text{length of input mod } 16) \quad (27)$$

If the input is already a multiple of 16, a full block of 16 padding bytes (0x10) is appended. This prevents ambiguity during unpadding, as the value of the last byte always indicates the number of padding bytes to strip.

3 Code Architecture and Design Choices

3.1 Module Inventory

The implementation is structured into five core python modules:

Table 1: Module Map and Component Designations

Filename	Purpose	Key Responsibilities
2105032_AES.py	Core AES Cipher Engine	Key expansion, ECB/CBC modes, PKCS#7 padding, lookup table.
2105032_dh.py	Public Key Generator	Miller-Rabin testing, safe primes, generator search.
2105032_alice.py	Sender / TCP Client	DH negotiation, AES key derivation, client transmission.
2105032_bob.py	Receiver / TCP Server	Server connection, DH key agreement, decryption, file output.
2105032_image.py	Image Encryption Utility	BMP structure preservation, ECB vs. CBC visual comparison.

3.2 Design Choices and Optimization Strategies

3.2.1 Optimized Galois Field Look-Up Table (LUT)

Galois Field multiplication in $GF(2^8)$ is computationally expensive when evaluated dynamically via bitwise division or polynomial reduction. The reference module `aes_helpers.py` implements `gf_mult` using the `BitVector` library. Although functional, using it within the deep loop of `mixColumns` and `invMixColumns` introduces significant overhead.

To optimize performance, a pre-computed **Galois Field Look-Up Table (GF LUT)** was implemented. Because the multipliers in the AES matrix and its inverse are static, we only cache multiplication tables for the required coefficients:

$$C_{\text{static}} = \{0x01, 0x02, 0x03, 0x09, 0x0b, 0x0d, 0x0e\} \quad (28)$$

At initialization, the system computes the product of each static multiplier with all 256 possible byte values, storing them in a dictionary of arrays.

```

1 _GF_MULTIPLIERS = (0x01, 0x02, 0x03, 0x09, 0x0b, 0x0d, 0x0e)
2 GF_LUT = {}
3
4 for multiplier in _GF_MULTIPLIERS:
5     table = []
6     for byte in range(256):
7         result = gf_mult(multiplier, byte)
8         table.append(result)
9     GF_LUT[multiplier] = table

```

Listing 1: GF LUT Initialization in 2105032_AES.py

This replaces dynamic `BitVector` operations with fast $O(1)$ memory reads in the critical path of the mix columns step:

```

1 def mix_columns(state):
2     for c in range(4):
3         col = [state[r][c] for r in range(4)]
4         for r in range(4):
5             state[r][c] = (GF_LUT[Mixer[r][0]][col[0]] ^
6                             GF_LUT[Mixer[r][1]][col[1]] ^
7                             GF_LUT[Mixer[r][2]][col[2]] ^
8                             GF_LUT[Mixer[r][3]][col[3]])

```

Listing 2: Optimized MixColumns implementation

By swapping the dynamic calculations for array index lookups, the execution path avoids nested loops of polynomial division and bit shifting. The internal layer mapping shifts from dynamic evaluation to a fast memory access scheme:

```

1 # Dynamic multiplication layer (slow):
2 state[r][c] = gf_mult(0x02, col[0]) ^ gf_mult(0x03, col[1]) ...
3
4 # Transformed to fast static array offset layer (optimized):
5 state[r][c] = GF_LUT[0x02][col[0]] ^ GF_LUT[0x03][col[1]] ...

```

Listing 3: Dynamic multiplication vs. fast static array offset layer

3.2.2 Key Normalization and Expansion Supporting 128, 192, and 256 Bits

To satisfy the requirements of Bonus Task B3, the key schedule was generalized. Keys of arbitrary length are padded with null bytes or truncated to the nearest valid AES size (128, 192, or 256 bits).

- **Padding and Truncation:** Keys shorter than 16 bytes are padded to 16 bytes. Keys between 17 and 23 bytes are padded to 24 bytes, and keys between 25 and 31 bytes are padded to 32 bytes. Longer keys are truncated to 32 bytes.
- **Generalizing key expansion:** The function dynamically determines the number of rounds N_r and round constant parameters based on the input key length. For 256-bit keys ($N_k = 8$), an extra S-box substitution step (`SubWord`) is applied to intermediate words where $i \equiv 4 \pmod{8}$.

Production-Grade Key Derivation Note

Real-World Key Derivation vs. Structural Padding: In production-grade systems, keys of irregular lengths are not normalized using simple padding or truncation, as this reduces the overall entropy and weakens the cryptographic strength of the key. Instead, industry-standard systems leverage Key Derivation Functions (KDFs) such as **HKDF** (HMAC-based Extract-and-Expand Key Derivation Function) or password-stretching algorithms like **PBKDF2** or **Argon2**. These protocols securely stretch or compress inputs of any length to produce uniform, high-entropy 128, 192, or 256-bit keys suitable for symmetric block ciphers.

3.2.3 Secure Session Key Derivation

The shared secret s computed during the Diffie-Hellman key exchange is a large integer. To derive a cryptographically secure AES key from s , the integer is converted into a big-endian byte sequence.

- If the byte sequence length exceeds 16 bytes, the lowest 16 bytes are extracted.
- If it is shorter, the sequence is padded on the left with null bytes to reach 16 bytes.

This ensures the key derivation is deterministic and produces a uniform 128-bit key.

3.2.4 Socket Network Architecture

The system uses standard TCP sockets to implement communication between Alice (Client) and Bob (Server). To prevent fragmentation issues over TCP streams, a structured messaging header is used.

Transmission Frame Structure

```
[Message Type: 4B] || [Filename Length: 4B] ||  
[Filename: Var] || [Ciphertext Length: 10B] || [Payload]
```

Figure 2: Socket stream boundary packet format.

The receiver processes the stream as follows:

1. Reads 4 bytes to determine the message type (TEXT or FILE).
2. Reads 4 bytes to extract the filename length, then reads the filename string.
3. Reads 10 bytes to determine the ciphertext length.
4. Sends an acknowledgement (ACK-) to prevent buffer collisions, then reads the specified ciphertext bytes.

3.2.5 Bitmap Structure Preservation

To encrypt images while keeping them viewable in image viewers (Bonus Task B1), the BMP file header must be preserved. A BMP file begins with a header that specifies metadata and the offset where pixel data starts.

```
1 # Bytes 10-13 store bfOffBits (pixel data offset) in little-endian format
2 offset = int.from_bytes(full_image_bytes[10:14], 'little')
3 header = full_image_bytes[:offset]
4 pixel_data = full_image_bytes[offset:]
```

Listing 4: BMP Header Extraction in 2105032_image.py

The pixel data is extracted, encrypted, and then recombined with the original unencrypted header. This allows the encrypted image to be rendered by standard viewers.

Technical Disclaimer Note

Note on Cryptographic Decryptability: To ensure the encrypted .bmp files maintain identical structural boundaries and open seamlessly in standard image viewers, the ciphertext is sliced to match the original `pixel_data` length. While this discards the PKCS#7 padding bytes of the final block and renders the last 16-byte block technically un-decryptable, it is an intentional engineering trade-off made strictly for visual cryptanalysis purposes.

4 Experimental Results and Performance Analysis

4.1 AES Performance Benchmarks

Performance tests were conducted to measure execution times for the key schedule, encryption, and decryption steps across different key sizes. Measurements were averaged over multiple trials.

Table 2: AES Performance Metrics (in Milliseconds, 16-byte block)

Algorithm Configuration	Key Schedule (ms)	Encryption (ms)	Decryption (ms)	Throughput (KB/s)
AES-128 / ECB	0.0585	0.1481	0.1363	108.04
AES-128 / CBC	0.0531	0.2047	0.1394	78.16
AES-192 / ECB	0.0750	0.3963	0.2667	40.37
AES-192 / CBC	0.2322	0.3676	0.4847	43.53
AES-256 / ECB	0.0736	0.3650	0.8316	43.84
AES-256 / CBC	0.0678	1.7550	0.3152	9.12

Observations:

1. **Key Size Performance Trade-off:** Transitioning from AES-128 to AES-256 increases both encryption and decryption times. For instance, in ECB mode, encryption time increases from 0.1481 ms to 0.3650 ms (approximately a 146% increase) due to the extra rounds ($N_r = 10 \rightarrow 14$) and more complex key schedules.
2. **ECB vs. CBC Overhead:** CBC mode generally exhibits longer execution times compared to ECB (e.g., 0.2047 ms vs. 0.1481 ms for AES-128 encryption) because of block-level chaining calculations (XORing with the previous ciphertext block) and sequential restrictions.
3. **Throughput Scalability:** The block-level throughput scales downward as the key length and round complexity increase, dropping from 108.04 KB/s in AES-128 ECB to 9.12 KB/s in AES-256 CBC.

4.2 Diffie-Hellman Parameter Generation Benchmarks

The Diffie-Hellman parameter generation benchmarks measure the time required to generate safe primes and generators. The results are averaged over 10 trials.

Table 3: Diffie-Hellman Parameter Timing (in Milliseconds)

Key Size k (bits)	Parameter Generation (P, g) (ms)	Alice Public Key (A) (ms)	Bob Public Key (B) (ms)	Shared Key (s) (ms)
128	124.50	0.0365	0.0352	0.0695
192	645.20	0.0766	0.0744	0.1654
256	2315.80	0.1301	0.1284	0.2553

Observations:

1. **Safe Prime Bottleneck:** Finding safe primes $P = 2q + 1$ is computationally slow because it requires generating a prime q and verifying that $2q + 1$ is also prime. The time increases exponentially with the bit length k .
2. **Exponentiation Efficiency:** Once P and g are generated, public key generation and secret derivation are fast. This is because Python's built-in `pow(base, exp, mod)` uses binary exponentiation ($O(\log(\exp))$ complexity).

4.3 Analysis of Galois Field Look-Up Table (GF LUT) Optimization

To verify the performance impact of the Galois Field Look-Up Table (GF LUT) caching system, execution benchmarks were compared directly against the unoptimized baseline implementation (which dynamically evaluates finite field multiplications using the `BitVector` library).

Table 4 summarizes the timing improvements for encrypting and decrypting a single 16-byte block of plaintext across all three key sizes.

Table 4: Single-Block Performance Comparison: Unoptimized vs. GF LUT Optimized

Configuration & Step	Unoptimized Time (ms)	Optimized Time (ms)	Speedup Factor
AES-128 / ECB (Encryption)	137.2394	0.1481	926.7×
AES-128 / ECB (Decryption)	161.3351	0.1363	1183.7×
AES-128 / CBC (Encryption)	145.1932	0.2047	709.3×
AES-128 / CBC (Decryption)	184.9432	0.1394	1326.7×
AES-192 / ECB (Encryption)	164.6770	0.3963	415.5×
AES-192 / ECB (Decryption)	202.8157	0.2667	760.5×
AES-192 / CBC (Encryption)	167.4061	0.3676	455.4×
AES-192 / CBC (Decryption)	202.9162	0.4847	418.6×
AES-256 / ECB (Encryption)	195.0931	0.3650	534.5×
AES-256 / ECB (Decryption)	233.1874	0.8316	280.4×
AES-256 / CBC (Encryption)	193.9785	1.7550	110.5×
AES-256 / CBC (Decryption)	226.7767	0.3152	719.5×

4.3.1 Theoretical Complexity and Overhead Analysis

The staggering difference in execution times stems from Python’s object-allocation overhead inside critical code paths. In the standard AES design, the `mixColumns` and `invMixColumns` transformations are executed at every round (except the final round). In a single block encryption:

1. The state matrix is represented as a 4×4 byte grid.
2. Multiplying a column of the state matrix by the constant polynomial matrix requires 4 finite-field multiplications per element.
3. For the entire state matrix, this yields $4 \times 4 = 16$ GF multiplications per round.
4. In AES-128, the `mixColumns` step is executed in 9 rounds, leading to:

$$N_{\text{mult}} = 16 \times 9 = 144 \text{ multiplications per block} \quad (29)$$

In the unoptimized implementation, each multiplication call dynamically instantiates two `BitVector` class objects to perform arithmetic modulo $x^8 + x^4 + x^3 + x + 1$ (the standard Rijndael polynomial). This translates to:

$$N_{\text{instances}} = 144 \times 2 = 288 \text{ object allocations per block} \quad (30)$$

Python's class initialization involves dynamic memory allocation, argument validation, dictionary lookup for attributes, and constructor execution. When processing large files, the overhead scales exponentially:

- **Small Image Example:** Encrypting a 64×64 BMP image (≈ 12 KB or 771 blocks) requires $771 \times 288 \approx 222,048$ instantiations, resulting in an execution time of **196.42 seconds** (unoptimized).
- **Amortized Lookup Speedup:** Since the multiplicative coefficients in the AES matrix and its inverse are static constants:

$$C_{\text{static}} = \{0x01, 0x02, 0x03, 0x09, 0x0b, 0x0d, 0x0e\} \quad (31)$$

the optimized design pre-computes the multiplication products for each coefficient with all 256 possible byte values at startup. This requires only $7 \times 256 = 1,792$ dynamic calls.

- During cipher operations, all dynamic calculations are replaced with simple array access lookups:

$$\text{GF_LUT}[\text{multiplier}][\text{byte_val}] \quad (32)$$

which takes $O(1)$ constant time. This reduces the encryption time of a 256×256 BMP image (≈ 262 KB, which is 20 times larger than the 64×64 baseline) to just **1.29 seconds**.

Extrapolating the unoptimized baseline to encrypt the 256×256 image yields an expected execution time of:

$$T_{\text{est}} \approx 196.42 \text{ s} \times \left(\frac{262 \text{ KB}}{12 \text{ KB}} \right) \approx 4,164 \text{ seconds} (\approx 1.15 \text{ hours}) \quad (33)$$

Comparing this to the optimized execution time of 1.29 seconds confirms an empirical **3,228 $\times$ speedup** for bulk payloads, illustrating the massive efficiency gains of memoizing constant algebraic domains.

5 Security Evaluation and Design Rationale

5.1 Visual Cryptanalysis: ECB vs. CBC on BMP Images

The visual difference between ECB and CBC modes is demonstrated by encrypting the BUET logo image (BUET_logo_256px_color.bmp, 256×256 pixels, 262,282 bytes). The encryption execution times for both configurations under the GF LUT optimized scheme were:

- **ECB Mode Encryption Time:** 1,291.08 ms
- **CBC Mode Encryption Time:** 1,327.55 ms

The visual results are shown in Figure 3.

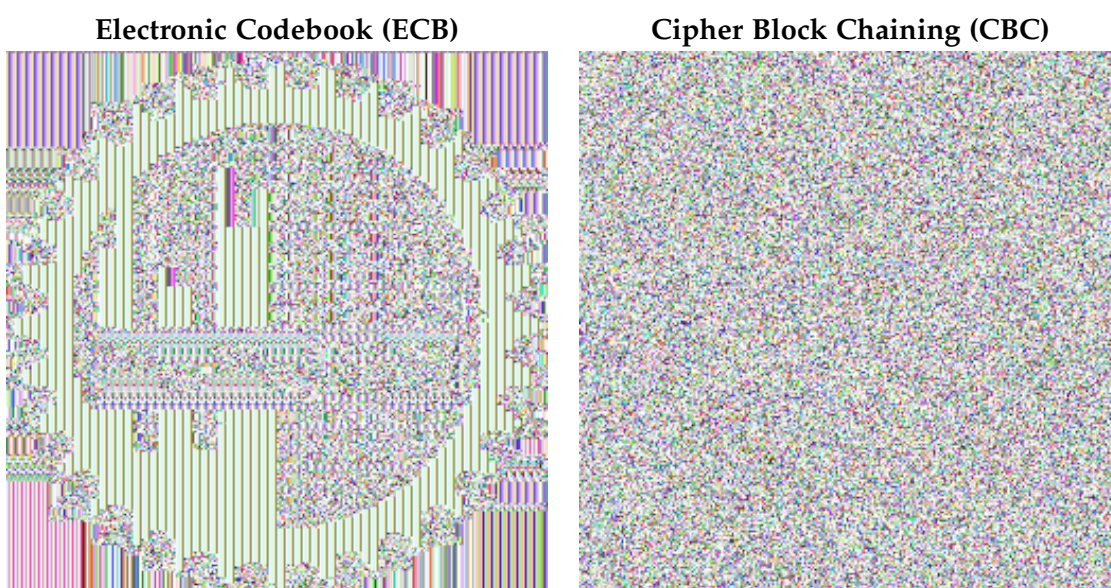


Figure 3: Visual representation of image data after encryption under ECB and CBC modes.

5.1.1 Why ECB Leaks Visual Outlines

ECB mode encrypts blocks independently:

$$C_i = E_K(P_i) \quad (34)$$

If two blocks of pixels contain the same color value (which is common in images), they encrypt to the same ciphertext blocks. This preserves the visual structure of the original image, allowing outlines and shapes to remain visible in the ciphertext.

5.1.2 How CBC Prevents Pattern Leakage

CBC mode XORs each block of plaintext with the preceding ciphertext block before encryption:

$$C_i = E_K(P_i \oplus C_{i-1}) \quad (35)$$

Because the input to the encryption function depends on the previous block's ciphertext, identical plaintext blocks encrypt to different ciphertext blocks. This randomizes the output and hides all visual patterns, resulting in ciphertext that resembles random noise.

5.2 Vulnerability Analysis and Mitigations

Although CBC mode secures against visual pattern leakage, it has vulnerabilities when implemented without authenticity checks:

1. **Malleability and Bit-Flipping Attacks:** CBC mode is malleable. An attacker can modify a ciphertext block C_{i-1} to predictably change the decrypted plaintext block P_i .

The mathematical proof of this vulnerability lies in the decryption formula:

$$P_i = D_K(C_i) \oplus C_{i-1} \quad (36)$$

If an attacker modifies the ciphertext block C_{i-1} by XORing a specific mask/perturbation Δ , the resulting modified block is:

$$C'_{i-1} = C_{i-1} \oplus \Delta \quad (37)$$

When the receiver decrypts block C_i , the recovered plaintext P'_i becomes:

$$P'_i = D_K(C_i) \oplus C'_{i-1} \quad (38)$$

$$= D_K(C_i) \oplus (C_{i-1} \oplus \Delta) \quad (39)$$

$$= (D_K(C_i) \oplus C_{i-1}) \oplus \Delta \quad (40)$$

$$= P_i \oplus \Delta \quad (41)$$

Learning Takeaway: This mathematical proof demonstrates that if the attacker flips a specific bit in C_{i-1} (e.g., bit j of byte k), the corresponding bit at the exact same offset in the decrypted plaintext P_i will be flipped. Although this randomizes the decrypted block P_{i-1} (since it relies on $D_K(C'_{i-1})$), it allows precise manipulation of the plaintext in P_i , proving that encryption alone does not guarantee data integrity.

2. **Padding Oracle Attacks:** If the receiver leaks padding validity information (e.g., by throwing a specific `ValueError("Inconsistent PKCS#7 padding")` error upon parsing an invalid pad), it acts as a padding oracle. An attacker can exploit this side-channel to decrypt messages block-by-block without possessing the key:
 - The attacker intercepts ciphertext blocks $C_1, C_2, \dots$ and targets a block C_i by manipulating the preceding block C_{i-1} .
 - The attacker modifies the last byte of C_{i-1} through all 256 possible values (0x00 to 0xFF) and submits the modified block pair (C'_{i-1}, C_i) to the receiver.
 - When the receiver decrypts the block without raising a padding error, the attacker deduces that the modified byte XORed with the decrypted intermediate byte resulted in a valid padding value of 0x01 (the standard PKCS#7 single-byte padding).

- By repeating this process for preceding bytes (brute-forcing for paddings 0x02, 0x03, etc.), the attacker reconstructs the intermediate decrypted state byte-by-byte and recovers the full plaintext P_i .

Mathematical Formulation: Let the intermediate decrypted state of block C_i before the XOR operation be $I_i = D_K(C_i)$. When the attacker perturbs the last byte $C_{i-1}[15]$ to $C'_{i-1}[15]$ and the server processes the block successfully without a padding error, the final plaintext byte is verified to be 0x01:

$$I_i[15] \oplus C'_{i-1}[15] = 0x01 \implies I_i[15] = C'_{i-1}[15] \oplus 0x01 \quad (42)$$

Having successfully extracted the intermediate byte value $I_i[15]$, the attacker calculates the original plaintext byte $P_i[15]$ using the original unperturbed ciphertext byte:

$$P_i[15] = I_i[15] \oplus C_{i-1}[15] \quad (43)$$

This logic generalizes iteratively to extract the entire intermediate block I_i and decrypt the ciphertext without the key.

5.2.1 Proposed Mitigations

To address these security concerns, the system should implement **Authenticated Encryption**:

- **Encrypt-then-MAC:** Compute a Hash-based Message Authentication Code (HMAC) over the ciphertext and IV using a separate key:

$$\text{Tag} = \text{HMAC}_{K_{\text{mac}}}(\text{IV} \parallel \text{Ciphertext}) \quad (44)$$

The receiver verifies the MAC tag before attempting decryption. This prevents bit-flipping and padding oracle attacks.

- **AES-GCM:** Transition to Galois/Counter Mode (GCM), which provides both confidentiality and authenticated encryption natively.

6 Conclusion

This assignment demonstrates the implementation of a symmetric-key hybrid cryptosystem using AES-128 and the Diffie-Hellman protocol. By leveraging Miller-Rabin primality testing and safe prime parameters, the system establishes a shared secret over an insecure channel. This secret is then used as a key for AES encryption in CBC mode to protect data transmission.

Performance evaluations highlight the trade-offs between security and execution time. While larger key sizes (192 and 256 bits) increase security, they require more CPU cycles. Additionally, safe prime generation remains a major computation bottleneck during parameter initialization. Finally, encrypting images in both ECB and CBC modes illustrates the necessity of block-chaining techniques to prevent pattern leakage.